

Research Article

Petr Babkin and Oleg Bakhteev

Structure Aware Neural Architecture Search for Mixture of Experts

<https://doi.org/10.1515/sample-YYYY-XXXX>

Received Month DD, YYYY; revised Month DD, YYYY; accepted Month DD, YYYY

Abstract: The Mixture-of-Experts (MoE) layer, a sparsely activated neural architecture controlled by a routing mechanism, has recently achieved remarkable success across large-scale deep learning tasks. In parallel, Neural Architecture Search (NAS) has emerged as a powerful methodology for automatically discovering high-performing neural network. However, the application of NAS methods to MoE architectures remains an underexplored research area. In this work, we propose an architecture search framework for MoE models, which explicitly leverages the underlying cluster structure of the data. We evaluate the proposed approach on computer vision benchmarks and demonstrate that it outperforms baseline MoE architectures trained on the same datasets in terms of accuracy and computational efficiency.

Keywords: Mixture of Experts, Neural Architecture Search

1 Introduction

The Mixture-of-Experts (MoE) architecture, in which input is partitioned through a gating function and subsequently routed as weighted input to specialized experts [1–3], has been well-established in the literature for decades [4]. Scenarios in which the experts are classical statistical models are well characterized in the literature [5], particularly when the experts are modeled as logistic regressors [6] or as Gaussian models [7, 8]. In recent years, concurrent with the growing interest in neural networks, researchers began exploring the application of MoE structures to deep learning architectures [9]. This efficiency was achieved through sparse gating mechanisms, where the input is routed only to a fixed number of experts, which is significantly smaller than the complete set [10–12]. MoE architectures found widespread adoption in the transformer domain, where model size plays a crucial role in achieving superior performance [13–15].

Simultaneously, the field of Neural Architecture Search (NAS) has developed into a well-studied area [16–18]. NAS aims to select optimal architectures from a wide range of candidates, addressing the challenge that architecture selection typically requires substantial domain expertise and manual tuning [16, 19]. The field has diverged into two main paradigms. The first is computationally intensive many-shot NAS, which involves multiple training runs followed by candidate selection using methods such as reinforcement learning [20, 21] and evolutionary algorithms [22, 23]. The second is one-shot NAS, where a single supernet is trained, and the optimal subnet is subsequently selected [19, 24, 25]. This approach effectively transforms the search problem into finding optimal compression strategies for the supernet [19].

Despite the individual successes of both MoE and NAS methodologies, the application of NAS techniques specifically to MoE architectures remains a relatively underexplored research area, presenting significant opportunities for advancing both efficiency and performance in large-scale neural architectures. In contrast to the classical approach where all experts share the same architecture [12], our study allows architectural heterogeneity between experts. We claim that this flexibility enhances the overall efficiency of the resulting architecture by enabling more specialized and adaptable expert networks.

It is important to note that several studies in the literature have addressed the problem of discovering or designing architectures for mixture-of-experts models. In the article [26], the authors employed a comprehensive combination of approaches to discover a fast-operating Mixture-of-Experts (MoE) architecture. The expert weights are derived from a supernet, and the search process is evolutionary in nature. There is a practical work [27] in which the authors develop a framework for efficient MoE inference tailored to specific hardware. The work most closely related to ours is [28], where the authors conduct a search by varying the number of experts in the layers. In contrast to this study, we propose to allow greater flexibility by optimizing the model architectures themselves rather than only the number of experts.

The MoE architecture is inherently well-suited for data with strong clustering properties. There exist theoretical studies, which demonstrate on relatively simple examples that this claim holds [29]. Additionally, practical researches show the effectiveness of MoE on multimodal datasets [30–32]. However, despite these promising results, the underlying mechanisms that enable experts to align with distinct modalities remain insufficiently understood, motivating further investigation into the principles governing modality-specific specialization within MoE frameworks.

Given the foregoing, we formulate the following research question: *how can we design conditions under which expert architectures adapt to specific data clusters?* In this study, we aim to address this question by examining how models evolve during adaptation to specific data modalities.

Contributions

- **Problem formulation.** We propose a new formulation of architecture search for Mixture-of-Experts models as a cluster-aware likelihood-maximisation problem, and give a theoretical grounding for it by showing its equivalence to the incomplete-data maximum likelihood of a latent-variable mixture.
- **Algorithm.** We propose an algorithm that solves the stated problem, based on the Expectation–Maximisation (EM) algorithm with an adaptively refined surrogate model.
- **Convergence guarantee.** We prove a convergence theorem for the proposed algorithm.
- **Benchmarks and experiments.** We construct datasets on which mixture-of-experts approaches have a clear advantage, and conduct comparative experiments against several baselines on them.

2 Related Work

Mixture-of-Experts. The Mixture-of-Experts (MoE) framework was originally introduced in [1–3] as a way to combine specialised statistical models through a learnable gating mechanism. Subsequent theoretical analyses characterised identifiability and convergence in classical MoE formulations [5–8]. With the rise of deep learning, MoE has been revived in the form of sparsely activated neural networks, where the gate routes each input to only a small subset of experts [9–12]. This sparse formulation enables training of models with hundreds of billions of parameters while keeping per-token compute roughly constant, and is now a standard building block of modern large-scale transformers [13–15].

Neural Architecture Search. Neural Architecture Search (NAS) automates the design of neural networks and has matured into a wide research field [16]. Many-shot approaches based on reinforcement learning [18, 20, 21] or evolutionary search [22, 23] explicitly train candidate architectures and select the best performer. One-shot methods reduce the search cost by training a single supernet that contains all candidate architectures as sub-networks [17, 19, 24, 25]. DARTS [19] is the most prominent representative of this family and forms one of the baselines we compare against in our experimental study.

Neural Architecture Search for Mixture-of-Experts. The intersection of NAS and MoE is a comparatively recent and still sparsely populated research area. AutoMoE [26] applies an evolutionary search on top of a shared supernet to obtain heterogeneous MoE architectures for neural machine translation. CMN [27] co-designs MoE topologies for computing-in-memory hardware. The work most closely related to ours is MoE-NAS [28], which optimises the number of experts per layer but keeps the expert architectures

shared. In contrast, we explicitly search over per-expert architectures and tie this search to the cluster structure of the input data.

Theoretical Understanding. A growing body of theoretical work studies why MoE models perform well on data with strong cluster or modality structure. [29] proves on simplified settings that MoE provably benefits from clusterable data, while [33] analyses the optimisation landscape of sparse gates. Empirical evidence from multimodal and multi-task learning [30–32, 34] further supports the claim that experts naturally specialise to distinct sub-distributions. Our work builds on this perspective and turns it into an explicit search objective by optimising over *per-cluster* expert architectures rather than treating the cluster structure as a by-product of training.

3 Problem Statement

3.1 Search Space of MoE

Each expert is a convolutional network assembled from a *cell*: a directed acyclic graph whose nodes are computation steps. We use the cell encoding of our implementation, in which a cell has a fixed number of nodes and each node applies a single operation, drawn from a fixed finite set $\mathcal{O}$ (convolutions of different kernel sizes, pooling, and a skip connection), to the output of one preceding node; the first node reads the cell input. An architecture is therefore determined by two discrete choices per node — *which* operation the node applies and *into which* node it sends that operation, i.e. which earlier node feeds into it — and we gather all these choices into a single architecture vector α , so that α describes one concrete cell. Since both $\mathcal{O}$ and the number of nodes are finite, the search space $\mathcal{A}$ of all admissible α is finite, and neural architecture search reduces to selecting the best $\alpha \in \mathcal{A}$. For an architecture α_k with trained weights θ_k we denote the resulting network by $f(\alpha_k, \theta_k) : \mathcal{X} \rightarrow \mathcal{Y}$.

A Mixture-of-Experts (MoE) model combines K such experts, each meant to specialise on a different region of the input space. Stacking the expert architectures into $\alpha = [\alpha_1^\top, \dots, \alpha_K^\top]^\top$, the MoE prediction is the routing-weighted combination of expert outputs

$$g(\alpha, \theta, \mathbf{w}_r)(\mathbf{x}) = \sum_{k=1}^K r_k(\mathbf{x}, \mathbf{w}_r) f(\alpha_k, \theta_k)(\mathbf{x}), \quad (1)$$

where the gate values $r_k(\mathbf{x}, \mathbf{w}_r) \geq 0$ with $\sum_{k=1}^K r_k(\mathbf{x}, \mathbf{w}_r) = 1$ are produced by a routing network with parameters $\mathbf{w}_r$.

Data clustering and routing. Prior to architecture search, the dataset is partitioned into M clusters, $\mathcal{D} = \mathcal{C}_1 \cup \mathcal{C}_2 \cup \dots \cup \mathcal{C}_M$, each grouping samples of similar structure (e.g. a semantic class group or a data modality). Throughout the paper, $n \in \{1, \dots, N\}$ indexes individual dataset samples, $m \in \{1, \dots, M\}$ indexes clusters, and $k \in \{1, \dots, K\}$ indexes experts. We write $\mathcal{C}_m \subseteq \{1, \dots, N\}$ for the sample indices that belong to cluster m , and denote by $m(n) \in \{1, \dots, M\}$ the cluster index of sample n .

Routing is defined at the cluster level: let $r_{mk} \in [0, 1]$ be the probability that cluster m is assigned to expert k , with $\sum_{k=1}^K r_{mk} = 1$, collected in a routing matrix $\mathbf{R} \in [0, 1]^{M \times K}$. The column $\mathbf{r}_k = \mathbf{R}_{:,k} \in [0, 1]^M$ encodes the clusters expert k is responsible for; the hard cluster $\rightarrow$ expert assignment recovered at the end of optimisation is $I_k = \{m : \arg \max_{k'} r_{mk'} = k\} \subseteq \{1, \dots, M\}$, the set of cluster indices routed to expert k .

Unlike standard MoE training, where only the gating parameters $\mathbf{w}_r$ are learned for a fixed architecture, here *finding the routing $\mathbf{R}$ is itself part of the architecture search*: the search jointly determines the per-expert architectures $\alpha_{1:K}$ and the assignment of clusters to experts. This couples architecture selection with data specialization, since the best architecture for an expert depends on the cluster subset it is trained on.

3.2 MoE Objective

We cast MoE architecture search as a likelihood-maximization problem. The goal is to choose, for every expert k , an architecture α_k , its trained parameters θ_k , and the cluster index set $I_k \subseteq \{1, \dots, M\}$ it is responsible for, so as to maximize the data likelihood under the mixture:

$$\prod_{n=1}^N \sum_{k=1}^K p(y_n, m(n) \in I_k \mid x_n, \alpha_k, \theta_k) \rightarrow \max_{\alpha_{1:K}, \theta_{1:K}, I_{1:K}}, \quad (2)$$

where the event $m(n) \in I_k$ encodes “sample n ’s cluster is routed to expert k ”.

Each joint term factorizes into a routing probability and a conditional likelihood:

$$p(y_n, m(n) \in I_k \mid x_n, \alpha_k, \theta_k) = p(m(n) \in I_k) p(y_n \mid x_n, \alpha_k, \theta_k, m(n) \in I_k). \quad (3)$$

The membership probability is precisely the cluster-level routing, $p(m(n) \in I_k) = r_{m(n),k}$. For the conditional factor we adopt the standard MLE proxy: since the experts are trained by minimising cross-entropy, the average per-sample negative log-likelihood (NLL) of expert k on its assigned data is exactly what training drives down. We therefore introduce a *surrogate function*

$$u : \mathcal{A} \times [0, 1]^M \rightarrow \mathbb{R}_{\geq 0}, \quad u(\alpha_k, \mathbf{r}_k) \approx -\frac{1}{|\mathcal{D}_{\mathbf{r}_k}^{\text{val}}|} \sum_{n \in \mathcal{D}_{\mathbf{r}_k}^{\text{val}}} \log p(y_n \mid x_n, \alpha_k, \theta_k^*), \quad (4)$$

that predicts the *mean per-sample cross-entropy loss* of architecture α_k trained on the cluster subset encoded by $\mathbf{r}_k$ (here θ_k^* are the parameters obtained by training α_k on $\mathcal{D}_{\mathbf{r}_k}$, and $\mathcal{D}_{\mathbf{r}_k}^{\text{val}}$ is the corresponding validation subset). A concrete construction of u as a learned regressor is given in Section 4.1.

The surrogate is treated as constant within a cluster: for every n in cluster m assigned to expert k ,

$$p(y_n \mid x_n, \alpha_k, \theta_k, m(n) \in I_k) \approx e^{-u(\alpha_k, \mathbf{r}_k)}, \quad (5)$$

i.e. u is the per-sample geometric mean of the conditional likelihood. Substituting (3) and (5) into (2) yields the surrogate-based objective

$$\prod_{n=1}^N \sum_{k=1}^K r_{m(n),k} e^{-u(\alpha_k, \mathbf{r}_k)} \rightarrow \max_{\alpha_{1:K}, \mathbf{r}_{1:K}}. \quad (6)$$

Because routing is decided at the cluster level, $r_{m(n),k} = r_{m,k}$ for every $n \in \mathcal{C}_m$, and $u(\alpha_k, \mathbf{r}_k)$ is identical across all samples of a cluster. Taking the logarithm of (6) and grouping the N per-sample terms by cluster collapses each cluster’s $|\mathcal{C}_m|$ identical log-factors into a single weighted term, yielding the *cluster-aware MoE objective*:

$$\mathcal{J}(\alpha_{1:K}, \mathbf{R}) = \sum_{m=1}^M |\mathcal{C}_m| \log \sum_{k=1}^K r_{mk} e^{-u(\alpha_k, \mathbf{r}_k)} \rightarrow \max_{\alpha_{1:K}, \mathbf{R}}. \quad (7)$$

Intuitively, r_{mk} specifies how strongly cluster m is routed to expert k , $e^{-u(\alpha_k, \mathbf{r}_k)}$ is the per-sample likelihood that expert k would attain on its assigned cluster subset, and the cluster-size weight $|\mathcal{C}_m|$ reflects that a cluster’s contribution to the joint log-likelihood scales with the number of samples it contains. Problem (7) is solved by the algorithm of Section 4.2 via Generalised EM with an adaptively refined surrogate.

4 Main Part

4.1 Surrogate Function

Direct optimisation of Problem 2 requires repeatedly training neural networks for many architecture candidates, which is prohibitively expensive in the MoE setting where each evaluation already involves K networks. To address this computational bottleneck we replace exact training by a learned surrogate model.

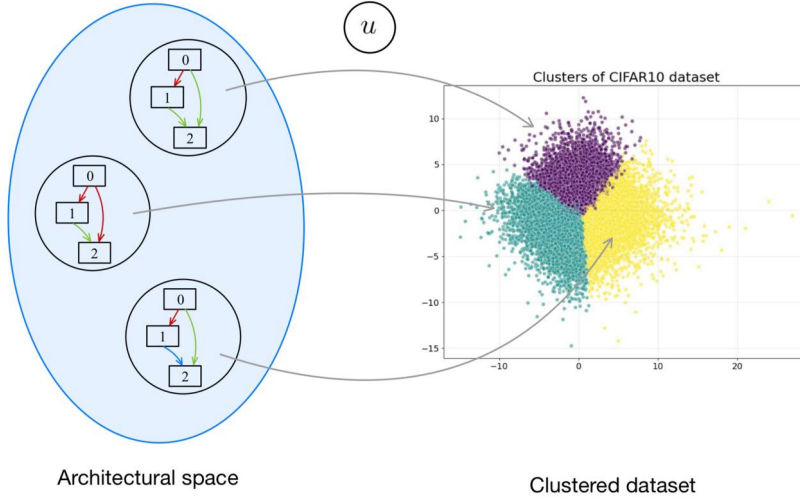


Fig. 1: Method workflow: a surrogate model u is used to select the most suitable expert architectures for each cluster.

Definition 1. Let the training data be partitioned into M clusters $\mathcal{D}_{\text{train}} = \mathcal{C}_1 \cup \dots \cup \mathcal{C}_M$, and for $\mathbf{b} \in \{0, 1\}^M$ let $\mathcal{D}_{\mathbf{b}} = \bigcup_{m:\mathbf{b}_m=1} \mathcal{C}_m$ be the cluster subset selected by $\mathbf{b}$. A surrogate function

$$u : \mathcal{A} \times \{0, 1\}^M \rightarrow \mathbb{R}_{\geq 0}, \quad u(\boldsymbol{\alpha}, \mathbf{b}) \approx \text{CE}_{\text{val}}(f(\boldsymbol{\alpha}, \boldsymbol{\theta}_{\boldsymbol{\alpha}, \mathbf{b}}^*)), \quad (8)$$

predicts the mean per-sample cross-entropy loss (i.e. the negative mean log-likelihood of the correct class) of architecture $\boldsymbol{\alpha}$ when trained on $\mathcal{D}_{\mathbf{b}}$, without performing the actual training. This is the same quantity that enters the MoE objective (7) through $e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)}$.

The surrogate is trained on a dataset of architecture/subset pairs collected by genuine training:

$$\mathcal{D}_{\text{surr}} = \{(\boldsymbol{\alpha}^{(i)}, \mathbf{b}^{(i)}, y^{(i)})\}_{i=1}^N, \quad y^{(i)} = \text{CE}_{\text{val}}(f(\boldsymbol{\alpha}^{(i)}, \boldsymbol{\theta}_{\boldsymbol{\alpha}^{(i)}, \mathbf{b}^{(i)}}^*)). \quad (9)$$

We parametrise $u_{\boldsymbol{\theta}}$ as a Graph Neural Network encoder over the architecture DAG concatenated with an embedding of the cluster mask $\mathbf{b}$, and minimise the mean squared regression loss

$$\min_{\boldsymbol{\theta}} \frac{1}{|\mathcal{D}_{\text{surr}}|} \sum_{(\boldsymbol{\alpha}, \mathbf{b}, y) \in \mathcal{D}_{\text{surr}}} (u_{\boldsymbol{\theta}}(\boldsymbol{\alpha}, \mathbf{b}) - y)^2. \quad (10)$$

The surrogate is refined inside an active learning loop. At every iteration we use Monte-Carlo dropout to obtain a predictive mean $\mu(\boldsymbol{\alpha}, \mathbf{b})$ and standard deviation $\sigma(\boldsymbol{\alpha}, \mathbf{b})$ over T stochastic forward passes, and score candidates by the Lower Confidence Bound $\text{LCB} = \mu - \sigma$ (since u is a loss, the most promising candidates are those with the *lowest* predicted CE together with high epistemic uncertainty). The most promising candidate is trained from scratch, the resulting triple is appended to $\mathcal{D}_{\text{surr}}$, and the surrogate is refit. The crucial property of $u_{\boldsymbol{\theta}}$ is that it is trained *only on discrete* masks $\mathbf{b}$; differentiable optimisation must therefore be combined with a relaxation of the discrete inputs.

4.2 Surrogate-assisted Generalized EM (SGEM)

Treating the cluster $\rightarrow$ expert routing as a latent variable turns the surrogate objective (7) into a mixture model on which Expectation–Maximisation applies directly, with the surrogate $u^{(t)}$ standing in for the otherwise-intractable expert quality. The derivation is the standard EM treatment of a finite mixture, the per-component data-fit term being the surrogate factor $e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)}$.

Latent-variable form of the objective. For every sample n introduce a one-hot latent vector $\mathbf{z}_n \in \{0, 1\}^K$ with $z_{nk} = 1 \iff$ sample n is generated by expert k , and collect them into the latent matrix

$\mathbf{Z} = \|z_{nk}\| \in \{0, 1\}^{N \times K}$. The generative story is the standard mixture one: the expert of sample n is drawn from its cluster's routing distribution, $p(z_{nk} = 1) = r_{m(n),k}$, and the label is then emitted with that expert's per-sample likelihood $e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)}$ [cf. (5)]. The resulting *complete-data* likelihood is

$$p(\mathbf{y}, \mathbf{Z} \mid \boldsymbol{\alpha}_{1:K}, \mathbf{R}) = \prod_{n=1}^N \prod_{k=1}^K \left(r_{m(n),k} e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)} \right)^{z_{nk}}, \quad (11)$$

in which the routing r_{mk} acts as the mixing weight and $e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)}$ as the per-component likelihood.

Equivalence with (7). Each row $\mathbf{z}_n$ is one-hot, so marginalising it collapses the inner product into a sum, $\sum_{\mathbf{z}_n} \prod_k (\cdot)^{z_{nk}} = \sum_k r_{m(n),k} e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)}$. Because routing and the surrogate are constant within a cluster, the $|\mathcal{C}_m|$ samples of cluster m contribute identical factors, and grouping them gives the incomplete-data likelihood

$$p(\mathbf{y} \mid \boldsymbol{\alpha}_{1:K}, \mathbf{R}) = \prod_{m=1}^M \left(\sum_{k=1}^K r_{mk} e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)} \right)^{|\mathcal{C}_m|}, \quad (12)$$

whose logarithm is *exactly* the cluster-aware objective $\mathcal{J}(\boldsymbol{\alpha}_{1:K}, \mathbf{R})$ of (7). Maximising (7) is thus the incomplete-data MLE of the latent-variable model (11), which we now solve by alternating maximisation of the variational free energy $\mathcal{F}_u(q, \theta) = \mathbb{E}_q[\log p(\mathbf{y}, \mathbf{Z} \mid \theta)] + \mathbb{H}[q]$ over the posterior $q(\mathbf{Z})$ (E-step) and the parameters $\theta = (\boldsymbol{\alpha}_{1:K}, \mathbf{R})$ (M-step).

E-step. At fixed $(\boldsymbol{\alpha}_{1:K}, \mathbf{R})$ the free energy is maximised by the exact posterior $q(\mathbf{Z}) = p(\mathbf{Z} \mid \mathbf{y}, \boldsymbol{\alpha}_{1:K}, \mathbf{R})$. From (11), $\log q(\mathbf{Z}) \propto \sum_n \sum_k z_{nk} (\log r_{m(n),k} - u(\boldsymbol{\alpha}_k, \mathbf{r}_k))$, so the posterior factorises over samples and the responsibility of expert k for sample n is the normalised mixture weight $\gamma_{nk} \propto r_{m(n),k} e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)}$. This depends on n only through its cluster $m(n)$, so all samples of a cluster share one responsibility, which we write as

$$q_{mk} = \frac{r_{mk} e^{-u(\boldsymbol{\alpha}_k, \mathbf{r}_k)}}{\sum_{j=1}^K r_{mj} e^{-u(\boldsymbol{\alpha}_j, \mathbf{r}_j)}}, \quad \sum_{k=1}^K q_{mk} = 1. \quad (13)$$

Intuitively, a cluster is softly attached to the experts that both already claim it (r_{mk} large) and are predicted to model it well ($u(\boldsymbol{\alpha}_k, \mathbf{r}_k)$ small).

M-step. Substituting the E-step posterior into $\mathbb{E}_q[\log p(\mathbf{y}, \mathbf{Z} \mid \cdot)]$ and grouping the sample-level terms by cluster yields the Q -function

$$Q(\boldsymbol{\alpha}_{1:K}, \mathbf{R}) = \sum_{m=1}^M |\mathcal{C}_m| \sum_{k=1}^K q_{mk} [\log r_{mk} - u(\boldsymbol{\alpha}_k, \mathbf{r}_k)], \quad (14)$$

which the M-step maximises over the routing $\mathbf{R}$ and the per-expert architectures $\boldsymbol{\alpha}_{1:K}$ (maximising Q at fixed q coincides with maximising $\mathcal{F}_u(q, \cdot)$, as the entropy $\mathbb{H}[q]$ is independent of θ). The two parameter blocks are updated in turn.

M-step (routing). For fixed architectures, Q is maximised over $\mathbf{R}$ subject to $\sum_k r_{mk} = 1$ by gradient ascent on the routing logits, using a straight-through Gumbel-Softmax to back-propagate through the discrete masks $\mathbf{r}_k$ that the surrogate consumes (recall u is trained only on binary $\mathbf{b}$). The term $\sum_{m,k} |\mathcal{C}_m| q_{mk} \log r_{mk}$ alone is a cross-entropy whose constrained optimum is $r_{mk} = q_{mk}$, i.e. it pulls the routing towards the current responsibilities, while the $-q_{mk} u(\boldsymbol{\alpha}_k, \mathbf{r}_k)$ term additionally biases each column $\mathbf{r}_k$ towards masks on which expert k is predicted to perform well.

M-step (architectures). For fixed routing the only architecture-dependent part of (14) is $-\sum_m |\mathcal{C}_m| q_{mk} u(\boldsymbol{\alpha}_k, \mathbf{r}_k)$, so each expert is optimised independently by *minimising* its responsibility-weighted surrogate loss. Since u is non-differentiable in $\boldsymbol{\alpha}$, we sample candidate architectures from the search space and keep, for each expert k , the candidate with the lowest $u(\boldsymbol{\alpha}_k, \mathbf{r}_k)$.

The complete procedure — alternating the E-step, the two M-steps over the routing $\mathbf{R}$ and the architectures $\boldsymbol{\alpha}_{1:K}$, and an active-learning refit of the surrogate over T iterations — is summarised in Algorithm 1.

Algorithm 1 Surrogate-assisted Generalized EM (SGEM) for MoE Architecture Search

Require: Surrogate u_θ , search space $\mathcal{A}$, number of experts K , clusters M , EM iterations T .

- 1: Initialise architectures $\alpha_1, \dots, \alpha_K$ at random and logits $\ell \in \mathbb{R}^{M \times K} \sim \mathcal{N}(\mathbf{0}, I)$.
- 2: **for** $t = 1$ to T **do**
- 3: **E-step.** $q_{mk} \propto r_{mk} e^{-u(\alpha_k, \mathbf{r}_k)}$.
- 4: **M-step (r).** Gradient ascent on $\sum_m |\mathcal{C}_m| \sum_k q_{mk} [\log r_{mk} - u(\alpha_k, \mathbf{r}_k)]$ using straight-through Gumbel-Softmax for $\mathbf{r}_k$.
- 5: **M-step (α).** For each expert k , sample candidates and keep the one with the lowest $u(\cdot, \mathbf{r}_k)$.
- 6: **Active learning step.** Train selected candidates, update $\mathcal{D}_{\text{sur}}$, refit u_θ .
- 7: **end for**
- 8: **return** $\alpha_{1:K}$ and hard assignment $\hat{r}_{mk} = 1, [k = \arg \max_{k'} r_{mk'}]$.

4.3 Convergence Analysis

A natural concern with Algorithm 1 is that the surrogate $u^{(t)}$ used inside the E- and M-steps is itself updated across iterations, so the classical monotonicity argument for EM [35, 36] no longer applies directly. We show that, provided the surrogate error decays fast enough, SGEM still converges in the same sense as Generalised EM.

Let u_{true} denote the (unknown) ground-truth expert loss, i.e. the per-sample CE attained by training architecture α on the cluster subset $\mathbf{r}$. Define the per-iteration surrogate error directly in the loss scale,

$$\varepsilon_t = \sup_{\alpha \in \mathcal{A}, \mathbf{r} \in [0,1]^M} |u^{(t)}(\alpha, \mathbf{r}) - u_{\text{true}}(\alpha, \mathbf{r})|. \quad (15)$$

The parameter $\theta = (\alpha_{1:K}, \mathbf{R})$ lies on the compact set $\Theta = \mathcal{A}^K \times (\Delta_K)^M$; we write $N = \sum_m |\mathcal{C}_m|$ for the total sample count. The (size-weighted) true log-likelihood is

$$L_{\text{true}}(\theta) = \sum_{m=1}^M |\mathcal{C}_m| \log \sum_{k=1}^K r_{mk} e^{-u_{\text{true}}(\alpha_k, \mathbf{r}_k)}, \quad (16)$$

and the *true* Q-function is

$$Q_{\text{true}}(\theta; \theta') = \sum_{m=1}^M |\mathcal{C}_m| \sum_{k=1}^K w_{mk}(\theta') [\log r_{mk} - u_{\text{true}}(\alpha_k, \mathbf{r}_k)], \quad (17)$$

where $w_{mk}(\theta')$ are the responsibilities computed under u_{true} (Proposition 1 in Appendix A).

Assumption 1. *There exists $U_{\max} < \infty$ such that $0 \leq u_{\text{true}}(\alpha, \mathbf{r}), u^{(t)}(\alpha, \mathbf{r}) \leq U_{\max}$ for all $\alpha \in \mathcal{A}, \mathbf{r} \in [0, 1]^M, t \geq 0$.*

Assumption 2. *The surrogate errors are summable: $\sum_{t=0}^{\infty} \varepsilon_t < \infty$.*

Assumption 3. *The M-step of Algorithm 1 is generalised, i.e. for every t it returns $\theta^{(t+1)}$ with $\mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t)})$, where the variational free energy is*

$$\mathcal{F}_u(q, \theta) = \sum_{m=1}^M |\mathcal{C}_m| \sum_{k=1}^K q_{mk} \log \frac{r_{mk} e^{-u(\alpha_k, \mathbf{r}_k)}}{q_{mk}}.$$

Assumption 4. *For every $\alpha \in \mathcal{A}$ the map $\mathbf{r} \mapsto u_{\text{true}}(\alpha, \mathbf{r})$ is continuous on $[0, 1]^M$.*

Theorem 1 (Convergence of SGEM with adaptive surrogate). *Under Assumptions 1–4, the iterates $\{\theta^{(t)}\}$ produced by Algorithm 1 satisfy*

- (a) *the sequence $\{L_{\text{true}}(\theta^{(t)})\}$ converges;*
- (b) *every limit point θ^* is a stationary point of L_{true} on Θ , in the sense $Q_{\text{true}}(\theta; \theta^*) \leq Q_{\text{true}}(\theta^*; \theta^*)$ for every $\theta \in \Theta$.*

The proof, together with the auxiliary lemmas (uniform deviation of the ELBO, stability of the softmax responsibilities, and a quasi-monotonicity bound $L_{\text{true}}(\theta^{(t+1)}) \geq L_{\text{true}}(\theta^{(t)}) - 2N\varepsilon_t$ established via a Robbins–Siegmund argument), is given in Appendix A.

Remark 1. *The summability assumption 2 is the surrogate analogue of the diminishing step-size condition in stochastic EM [35]; replacing it by the weaker condition $\varepsilon_t \rightarrow 0$ preserves part (b) of Theorem 1 but not part (a). Practically, summability is achieved whenever the active learning step in Algorithm 1 provides sufficiently rich coverage of $\mathcal{A} \times [0, 1]^M$.*

Novelty and relation to classical convergence results. The classical convergence theory of EM and its generalised variant [35, 36] rests on the monotone ascent of a single, *fixed* likelihood: every M-step cannot decrease the same objective, so convergence follows from a monotone–bounded sequence. Our setting violates this premise. Because the quality of an expert is read off an adaptively re-fitted surrogate $u^{(t)}$ rather than from exact training, the objective optimised at iteration t is itself a moving target, and the true likelihood L_{true} we ultimately care about is *never* evaluated by the algorithm. Hence no exact monotonicity is available and the standard argument does not apply.

Theorem 1 shows that convergence is retained nonetheless, which is the main theoretical contribution. The mechanism is a *quasi*-monotonicity bound $L_{\text{true}}(\theta^{(t+1)}) \geq L_{\text{true}}(\theta^{(t)}) - 2N\varepsilon_t$: the true likelihood may dip, but only by a slack proportional to the per-iteration surrogate error ε_t . Summability of these errors turns $\{L_{\text{true}}(\theta^{(t)})\}$ into an almost-supermartingale, and a Robbins–Siegmund argument [37] yields both convergence and stationarity of every limit point of the *true* objective — even though each E- and M-step sees only the surrogate.

This contrasts with stochastic and online EM analyses [38], where the perturbation of the exact EM update is *zero-mean stochastic noise* (from Monte-Carlo E-steps) tamed by a diminishing step-size schedule. In our case the perturbation is instead a *deterministic, bounded approximation error* of a learned surrogate, and the role of the vanishing step size is played by the summability condition 2, whose smallness is secured not by averaging out noise but by the active-learning coverage of the search space. To our knowledge, this is the first convergence guarantee for an EM-type architecture-search procedure whose per-iteration objective is supplied by an *adaptively trained* surrogate model.

5 Experiments

We evaluate the proposed framework on two complementary problems: (i) a controlled 2D toy task that allows us to verify whether the recovered routing matches the ground-truth cluster structure, and (ii) the CIFAR-100 image classification benchmark, where we compare against several MoE and NAS baselines.

5.1 Toy Experiment

We sample 2000 points in $\mathbb{R}^2$ from a four-class problem consisting of one “linear” cloud (two classes separated by a diagonal) and one “ring” cloud (two classes corresponding to an inner and an outer ring). The data is split 80/20 into training/validation and clustered into $M = 20$ KMeans clusters, yielding the partition described in our data preparation pipeline. We use $K = 2$ experts so that the ideal solution is unambiguous: one expert should specialise to the linear cloud and the other to the ring cloud. The search space is a small cell-based space over the 2D inputs, in the spirit of [19], whose operation set is deliberately chosen so that the two clouds genuinely prefer *different* architectures. In particular it contains a *linear* operation — a linear map $\mathbf{x} \mapsto W\mathbf{x}$ that already suffices to separate the linearly-separable cloud, whose two classes are split by a straight diagonal — and a *radial basis function* (RBF) operation $\mathbf{x} \mapsto \exp(-\gamma \|\mathbf{x} - \mathbf{c}\|^2)$, which encodes the distance to a centre $\mathbf{c}$ and is naturally suited to the ring cloud, whose two classes (the inner and the outer ring) are separated by radius rather than by any hyperplane. A few auxiliary operations

(identity, element-wise squaring, additive shift, ReLU and dropout) complete the set. Since a purely linear architecture cannot model the concentric rings, while an RBF-dominated one is needlessly poor on the linear cloud, the optimal mixture is forced to assign *architecturally distinct* experts to the two regions; the toy task therefore probes architectural specialisation, not merely the routing.

With the parameters reported in our experimental log, SGEM (Algorithm 1) recovers the *ideal* cluster split (10, 10) between the two experts in 20 EM iterations, matching the hand-defined oracle assignment that splits clusters according to the centroid coordinate $x_0 = -2$. This sanity check confirms that the surrogate-guided objective in (7) is well posed and that the proposed optimiser solves it when the surrogate is sufficiently accurate, in agreement with the convergence guarantee of Theorem 1.

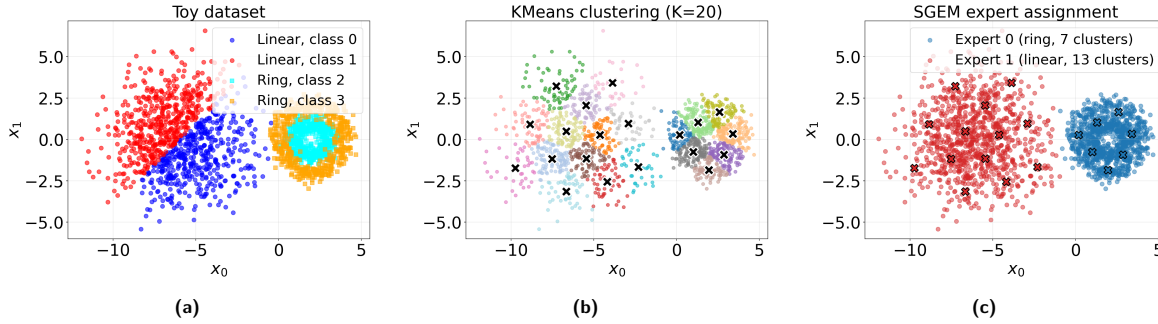


Fig. 2: Toy experiment. (a) The four-class dataset: a linear cloud (classes 0–1) and a ring cloud (classes 2–3). (b) The $M = 20$ KMeans clusters (crosses mark the centroids). (c) The expert assignment recovered by SGEM with $K = 2$: it splits the clusters exactly along the linear/ring boundary, matching the oracle partition up to a permutation of expert labels.

5.2 CIFAR-100 and SVHN mixture

To test whether the cluster-aware objective recovers a meaningful data partition on realistic data, we build a heterogeneous benchmark by mixing CIFAR-100 and SVHN in equal proportions (42 000 images total). SVHN labels are shifted by 100, so the combined problem has 110 classes. The two sources form a maximally separated domain structure, which gives a well-defined oracle: with $K = 2$ experts the ideal routing assigns each cluster to the expert specialised on its source. We partition the data into $M = 30$ *semantic* clusters obtained from a pretrained feature extractor, using a three-way train/validation/test split (0.7/0.15/0.15). KMeans is fit on the training embeddings only; validation and test points are assigned to the nearest centroid. The resulting oracle partition is 19 CIFAR-majority and 11 SVHN-majority clusters, with a mean source purity of 0.978.

All MoE configurations use $K = 2$ experts with `init_channels = 16`. The base network is a small DARTS-style architecture `CIFAR100Net` (stem, two normal cells, one fixed reduction cell, global average pooling, fully connected head). The MoE wrapper instantiates K such networks combined through a fixed hard *cluster gate*: each cluster is routed to a single expert, so the gate directly reflects the discovered data partition. Architecture search uses the train/validation split; the final model is retrained for 100 epochs on $\text{train} \cup \text{validation}$ and evaluated *once* on the held-out test set. We report test accuracy averaged over several random seeds (the number per method is given in Table 1). Training is performed inside a Docker container with PyTorch 2.5.1 on a single RTX 3080 Ti GPU.

Load balancing. Left unregularised, the EM dynamics of Algorithm 1 tend to collapse all clusters onto a single expert — a property of the objective itself rather than of the optimiser. At a hard routing, where every cluster m is sent to a single expert $k(m)$ (so $r_{mk} = [k = k(m)]$), the inner sum of (7) has a

single non-zero term and the logarithm cancels the exponential, giving

$$\mathcal{J} = \sum_{m=1}^M |\mathcal{C}_m| (-u(\boldsymbol{\alpha}_{k(m)}, \mathbf{r}_{k(m)})) = - \sum_{k=1}^K N_k u_k, \quad N_k = \sum_{m: k(m)=k} |\mathcal{C}_m|, \quad (18)$$

with $u_k = u(\boldsymbol{\alpha}_k, \mathbf{r}_k)$ the predicted loss of expert k and N_k the number of samples routed to it. If the surrogate becomes nearly constant in its routing argument and the experts are interchangeable ($u_k \equiv \bar{u}$), then $\mathcal{J} = -\bar{u} N$ is independent of the assignment $\{N_k\}$, so the likelihood is flat over the routing simplex; and even when the u_k differ, $\mathcal{J}$ is linear in the counts N_k and is maximised by routing all clusters to the single expert with the smallest u_k . To counteract this we augment the M-step of Algorithm 1 with a load-balancing penalty

$$\text{LB}(\mathbf{R}) = K \cdot \sum_{k=1}^K \bar{r}_k \cdot p_k, \quad \bar{r}_k = \frac{1}{M} \sum_{m=1}^M r_{mk}, \quad p_k = \frac{|\{m : \arg \max_{k'} r_{mk'} = k\}|}{M}, \quad (19)$$

weighted by a coefficient λ_{LB} ; it is minimal for a uniform routing and saturates at K under full collapse. Because the correct source partition is itself imbalanced (19/11), an aggressive penalty would push the routing *away* from it; we therefore keep λ_{LB} small ($\lambda_{\text{LB}} = 1$) and prevent collapse instead through the data-collection scheme described below. Appendix B.2 makes this trade-off quantitative, locating the cross-over at $\lambda_{\text{LB}} \approx 3.77$ beyond which a balanced — and hence domain-blind — split already scores higher on the penalised objective.

We compare against the following methods, all evaluated with the same cluster gate:

- **Random-MoE**: 200 random architecture pairs are searched at 30 epochs each on a random cluster split, then the best is retrained.
- **DARTS×K**: K experts that share the same DARTS-discovered architecture on a random cluster split.
- **SGEM** (ours): Algorithm 1 with a small load-balancing weight $\lambda_{\text{LB}} = 1$ and the two data-collection fixes described next. No source labels are used at any stage.
- **DARTS oracle MoE**: an *upper bound* in which the cluster→expert routing is fixed to the true source split and each expert uses a DARTS architecture searched on its own source. It uses the source labels and is therefore not a competitor but a ceiling on what perfect routing can achieve.

Honest data collection. An earlier version of our pipeline reached only ≈ 0.59 test accuracy with near-random source recovery ($\approx 60\%$), which we traced to two defects in how the surrogate dataset was built rather than to the EM optimiser. First, each candidate architecture was evaluated on the *entire* validation set instead of on the cluster region it was trained for, so an expert trained on, say, pure-SVHN clusters was scored partly on CIFAR images and looked poor; this collapsed the predicted-loss gap between a specialised and a mixed expert and erased the very signal the routing relies on. Restricting the evaluation to the expert’s own clusters (*region-restricted* surrogate targets) removes the bias. Second, the cluster masks used to train the surrogate were drawn by including each cluster independently with probability $\frac{1}{2}$, so a mask contained $\approx M/2$ clusters and a near-pure single-source subset was essentially never sampled (probability 2^{-19} for pure SVHN); the surrogate therefore never learned that a pure-source expert attains a low loss. Replacing this with a domain-agnostic *log-uniform* sampler over the mask size (minimum three clusters) makes small, often near-pure subsets frequent — without ever consulting the source labels. With both fixes and $\lambda_{\text{LB}} = 1$, SGEM recovers the source structure on its own.

Table 1 reports test accuracy with cluster gating. With the honest data collection, SGEM reaches 0.6199 ± 0.009 , outperforming both DARTS×K (0.5913 ± 0.009) and the random-MoE baseline (0.5467 ± 0.029) by +2.9 and +7.3 percentage points and coming within 1.3 p.p. of the DARTS oracle MoE (0.6327 ± 0.004) that is *given* the true source split. The cluster gate is what isolates routing quality: under a learnable gate the methods become nearly indistinguishable (all ≈ 0.63), because a trainable gate can compensate for a suboptimal partition and so masks the benefit of a good one. A finer ablation that decomposes this accuracy into a routing and an architecture contribution is reported in Appendix B.1: the gain stems almost entirely from the cluster→expert routing, while architectural heterogeneity between experts adds little once the



Fig. 3: Per-cluster representatives of the CIFAR-100 / SVHN mixture under the SGEM routing ($K = 2$, seed 322; first 15 of the $M = 30$ semantic clusters shown). Each row is one cluster: the leftmost column is the cluster medoid and the rest are the eight images nearest to the cluster mean. The medoid border is coloured by the expert SGEM assigns to the cluster (E0, the SVHN specialist, red; E1, the CIFAR specialist, green). SGEM sends every SVHN (digit) cluster to E0 and keeps E1 a pure CIFAR expert; the source-recovery rate is 93% for this seed.

routing is correct, and learning the routing inside SGEM (rather than freezing it) is what recovers the source structure.

Tab. 1: Test accuracy on the CIFAR-100 / SVHN mixture with cluster gating ($M = 30$, $K = 2$, semantic clustering, mean $\pm$ std). The DARTS oracle MoE uses the ground-truth source split and is an upper bound, not a competitor.

Method	test acc. (cluster gate)	seeds
Random-MoE	0.5467 ± 0.029	4
DARTS $\times K$ (same arch)	0.5913 ± 0.009	5
SGEM (ours)	0.6199 ± 0.009	3
DARTS oracle MoE (<i>uses source labels</i>)	0.6327 ± 0.004	3

Because the oracle partition by data source is sharp (mean purity 0.978), we can directly check whether the learned routing aligns with it. Across three seeds the SGEM hard assignment now agrees with the oracle source split on $92\% \pm 2\%$ of clusters (best of the two label permutations) — far above the $\approx 54\%$ of a balanced random split and a large jump over the $\approx 60\%$ of the earlier biased pipeline. In a typical run (seed 322, Figure 3) one expert becomes a *pure* CIFAR specialist (17 clusters, no SVHN) while the other absorbs all 11 SVHN clusters together with only 2 CIFAR ones. The cluster-aware objective therefore *does* discover the underlying data domains on this benchmark, reversing our earlier negative finding; the small residual

gap to the oracle stems from a few genuinely mixed clusters and from objective noise near the optimum (the best-objective iterate occasionally misses the 30/30 split attained one iteration earlier).

Convergence. Figure 4 traces a single SGEM run across its EM iterations. The size-weighted objective $L^{(t)}$ increases up to optimisation noise, while the source-recovery rate climbs from 67% to 100%, empirically illustrating the convergence guaranteed by Theorem 1. The two curves peak one iteration apart — the objective at $t = 8$, the recovery at $t = 9$ — the same objective/quality mismatch near the optimum noted above, so selecting the iterate by best objective returns a near-ideal but not always perfect split.

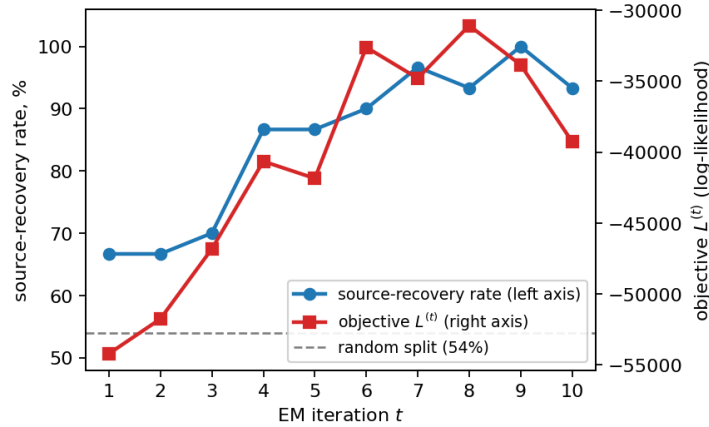


Fig. 4: Convergence of SGEM on the CIFAR-100/SVHN mixture (seed 322). Left axis (blue): agreement of the hard routing with the ground-truth source partition; the dashed line marks the 54% random-split level. Right axis (red): the size-weighted objective $L^{(t)}$. Both rise over the EM iterations, the source-recovery rate reaching 100% at $t = 9$.

6 Conclusion

We presented SA-NAS, a Structure-Aware Neural Architecture Search framework for Mixture-of-Experts models, which couples per-expert architecture search with the cluster structure of the underlying data. The framework relies on a Graph Neural Network surrogate that predicts validation accuracy from an architecture description and a binary cluster mask, trained inside an active-learning loop with Monte-Carlo dropout uncertainty estimates. Three optimisation algorithms (sampling, Gumbel-Softmax gradient and EM) instantiate the same cluster-aware MoE objective and share a common implementation of the surrogate.

On the controlled toy problem the EM variant recovers the ideal linear/ring split with $K = 2$ experts, confirming that the formulation is well posed when the surrogate is reliable. On a deliberately heterogeneous CIFAR-100 / SVHN mixture, whose ground-truth domains give a sharp oracle partition, SGEM recovers that partition on 92% of clusters *without ever seeing the source labels*: one expert specialises to a pure CIFAR sub-task while the other collects the SVHN clusters. Under the resulting hard cluster gate it attains 0.620 test accuracy, ahead of the random-MoE baseline (0.547) and of K copies of a single DARTS architecture (0.591), and within 1.3 p.p. of an oracle MoE (0.633) that is handed the true split. A diagnostic analysis was decisive: an earlier pipeline that scored experts on the whole validation set and sampled cluster masks uniformly never exposed the surrogate to near-pure subsets and collapsed the routing, whereas region-restricted evaluation together with a domain-agnostic log-uniform mask sampler removes this bias and lets the cluster-aware objective discover the data structure on its own.

These results are obtained on a two-domain benchmark with $K = 2$ and a modest number of seeds; broadening to more domains, larger K , and a wider seed sweep is the natural next step. We also plan to scale the search space, to extend the framework to transformer-based MoE layers, and to strengthen the

theory for the cluster-aware objective — in particular by folding the load-balancing term, which the present convergence analysis omits, into the guarantee.

References

- [1] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- [2] Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural computation*, 6(2):181–214, 1994.
- [3] Robert A Jacobs, Fengchun Peng, and Martin A Tanner. A bayesian approach to model selection in hierarchical mixtures-of-experts architectures. *Neural Networks*, 10(2):231–241, 1997.
- [4] Siyuan Mu and Sen Lin. A comprehensive survey of mixture-of-experts: Algorithms, theory, and applications. *arXiv preprint arXiv:2503.07137*, 2025.
- [5] Jakub Piwko, Jędrzej Ruciński, Dawid Płudowski, Antoni Zajko, Patrycja Żak, Mateusz Zacharecki, Anna Kozak, and Katarzyna Woźnica. Divide, specialize, and route: A new approach to efficient ensemble learning. *arXiv preprint arXiv:2506.20814*, 2025.
- [6] Huy Nguyen, Pedram Akbarian, TrungTin Nguyen, and Nhat Ho. A general theory for softmax gating multinomial logistic mixture of experts. *arXiv preprint arXiv:2310.14188*, 2023.
- [7] Huy Nguyen, TrungTin Nguyen, and Nhat Ho. Demystifying softmax gating function in gaussian mixture of experts. *Advances in Neural Information Processing Systems*, 36:4624–4652, 2023.
- [8] Carl Rasmussen and Zoubin Ghahramani. Infinite mixtures of gaussian process experts. *Advances in neural information processing systems*, 14, 2001.
- [9] David Eigen, Marc'Aurelio Ranzato, and Ilya Sutskever. Learning factored representations in a deep mixture of experts. *arXiv preprint arXiv:1312.4314*, 2013.
- [10] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [11] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. *Advances in Neural Information Processing Systems*, 34:8583–8595, 2021.
- [12] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [13] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [14] Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. *arXiv preprint arXiv:2501.11873*, 2025.
- [15] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- [16] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojian Chen, and Xin Wang. A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)*, 54(4):1–34, 2021.
- [17] Niv Nayman, Asaf Noy, Tal Ridnik, Itamar Friedman, Rong Jin, and Lihi Zelnik. Xnas: Neural architecture search with expert advice. *Advances in neural information processing systems*, 32, 2019.
- [18] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8697–8710, 2018.
- [19] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.
- [20] Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2016.
- [21] Bowen Baker, Otkrist Gupta, Nikhil Naik, and Ramesh Raskar. Designing neural network architectures using reinforcement learning. *arXiv preprint arXiv:1611.02167*, 2016.
- [22] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence*, volume 33, pages 4780–4789, 2019.
- [23] Zhaohui Yang, Yunhe Wang, Xinghao Chen, Boxin Shi, Chao Xu, Chunjing Xu, Qi Tian, and Chang Xu. Cars: Continuous evolution for efficient neural architecture search. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1829–1838, 2020.

- [24] Xiangning Chen and Cho-Jui Hsieh. Stabilizing differentiable architecture search via perturbation-based regularization. In *International conference on machine learning*, pages 1554–1565. PMLR, 2020.
- [25] Shan You, Tao Huang, Mingmin Yang, Fei Wang, Chen Qian, and Changshui Zhang. Greedynas: Towards fast one-shot nas with greedy supernet. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1999–2008, 2020.
- [26] Ganesh Jawahar, Subhabrata Mukherjee, Xiaodong Liu, Young Jin Kim, Muhammad Abdul-Mageed, Laks VS Lakshmanan, Ahmed Hassan Awadallah, Sebastien Bubeck, and Jianfeng Gao. Automoe: Heterogeneous mixture-of-experts with adaptive computation for efficient neural machine translation. *arXiv preprint arXiv:2210.07535*, 2022.
- [27] Shihao Han, Sishuo Liu, Shucheng Du, Mingzi Li, Zijian Ye, Xiaoxin Xu, Yi Li, Zhongrui Wang, and Dashan Shang. Cmn: a co-designed neural architecture search for efficient computing-in-memory-based mixture-of-experts. *Science China Information Sciences*, 67(10):200405, 2024.
- [28] Lotfi Abdelkrim Mecharbat, Alberto Marchisio, Muhammad Shafique, Mohammad M Ghassemi, and Tuka Alhanai. Moenas: Mixture-of-expert based neural architecture search for jointly accurate, fair, and robust edge deep neural networks. *arXiv preprint arXiv:2502.07422*, 2025.
- [29] Zixiang Chen, Yihe Deng, Yue Wu, Quanquan Gu, and Yuanzhi Li. Towards understanding the mixture-of-experts layer in deep learning. *Advances in neural information processing systems*, 35:23049–23062, 2022.
- [30] Tsz Chai Fung and Spark C Tseung. Mixture of experts models for multilevel data: modelling framework and approximation theory. *arXiv e-prints*, pages arXiv–2209, 2022.
- [31] Jiang Guo, Darsh J Shah, and Regina Barzilay. Multi-source domain adaptation with mixture of experts. *arXiv preprint arXiv:1809.02256*, 2018.
- [32] Zhiwei Hu, Víctor Gutiérrez-Basulto, Zhiliang Xiang, Ru Li, and Jeff Z Pan. Multi-level mixture of experts for multi-modal entity linking. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 979–990, 2025.
- [33] Hongbo Li, Sen Lin, Lingjie Duan, Yingbin Liang, and Ness B Shroff. Theory on mixture-of-experts in continual learning. *arXiv preprint arXiv:2406.16437*, 2024.
- [34] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35: 7103–7114, 2022.
- [35] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38, 1977.
- [36] C. F. Jeff Wu. On the convergence properties of the EM algorithm. *The Annals of Statistics*, 11(1):95–103, 1983.
- [37] Herbert Robbins and David Siegmund. A convergence theorem for non negative almost supermartingales and some applications. In *Optimizing Methods in Statistics*, pages 233–257. Academic Press, 1971.
- [38] Olivier Cappé and Éric Moulines. On-line expectation–maximization algorithm for latent data models. *Journal of the Royal Statistical Society, Series B*, 71(3):593–613, 2009.

A Proof of Theorem 1

This appendix collects the lemmas and the full proof of Theorem 1. We follow the notation of Section 4.3: $\theta = (\alpha_{1:K}, \mathbf{R}) \in \Theta = \mathcal{A}^K \times (\Delta_K)^M$ with $\mathcal{A}$ finite, ε_t is the per-iteration surrogate error (in the loss scale; see (15)), $N = \sum_m |\mathcal{C}_m|$, and the variational free energy parameterised by a loss function u is

$$\mathcal{F}_u(q, \theta) = \sum_{m=1}^M |\mathcal{C}_m| \sum_{k=1}^K q_{mk} \log \frac{r_{mk} e^{-u(\alpha_k, \mathbf{r}_k)}}{q_{mk}}.$$

We write $L_u(\theta) = \max_q \mathcal{F}_u(q, \theta)$ for the size-weighted log-likelihood induced by loss u , and abbreviate $L_{\text{true}} = L_{u_{\text{true}}}$, $L^{(t)} = L_{u^{(t)}}$.

A.1 Preliminary Lemmas

Proposition 1 (ELBO maximisation). *For every loss function $u : \mathcal{A} \times [0, 1]^M \rightarrow \mathbb{R}_{\geq 0}$ and every $\theta \in \Theta$, $L_u(\theta) = \max_{q \in \mathcal{Q}} \mathcal{F}_u(q, \theta)$ with the maximum attained at*

$$q_{mk}^*(\theta) = \frac{r_{mk} e^{-u(\alpha_k, \mathbf{r}_k)}}{\sum_j r_{mj} e^{-u(\alpha_j, \mathbf{r}_j)}}.$$

Proof. Fix θ and m . The factor $|\mathcal{C}_m|$ is a positive constant; it can be pulled out of the inner maximisation in $q_{m,:}$ without affecting the optimum. Writing $a_{mk} = r_{mk} e^{-u(\alpha_k, \mathbf{r}_k)}$ and using $\sum_k q_{mk} = 1$, the inner expression equals $-\text{KL}(q_{m,:} \| a_{m,:} / \sum_j a_{mj}) + \log \sum_j a_{mj}$. The KL-divergence is non-negative and vanishes exactly at $q_{mk} = a_{mk} / \sum_j a_{mj}$, which yields the claim. Summing over m with weights $|\mathcal{C}_m|$ recovers $L_u(\theta) = \sum_m |\mathcal{C}_m| \log \sum_k a_{mk}$. $\square$

A consequence is the Neal–Hinton decomposition $L_u(\theta) = \mathcal{F}_u(q, \theta) + \sum_m |\mathcal{C}_m| \text{KL}(q_{m,:} \| q_{m,:}^*(\theta))$, implying $L_u(\theta) \geq \mathcal{F}_u(q, \theta)$ with equality iff $q = q^*(\theta)$.

Lemma 1 (Uniform ELBO deviation). *Under Assumption 1, for every q, θ, t , $|\mathcal{F}_{u_{\text{true}}}(q, \theta) - \mathcal{F}_{u^{(t)}}(q, \theta)| \leq N \varepsilon_t$.*

Proof. The terms $\log r_{mk}$ and $-\log q_{mk}$ cancel between the two ELBOs, leaving

$$\mathcal{F}_{u_{\text{true}}} - \mathcal{F}_{u^{(t)}} = \sum_m |\mathcal{C}_m| \sum_k q_{mk} [u^{(t)}(\alpha_k, \mathbf{r}_k) - u_{\text{true}}(\alpha_k, \mathbf{r}_k)].$$

Bounding the inner bracket by ε_t and using $\sum_k q_{mk} = 1$ gives $|\mathcal{F}_{u_{\text{true}}} - \mathcal{F}_{u^{(t)}}| \leq \varepsilon_t \sum_m |\mathcal{C}_m| = N \varepsilon_t$. $\square$

Corollary 1. *For every θ , $|L_{\text{true}}(\theta) - L^{(t)}(\theta)| \leq N \varepsilon_t$.*

Proof. Apply Lemma 1 to the maxima of the two ELBOs and use $|\max_q f - \max_q g| \leq \sup_q |f - g|$. $\square$

Finally, we record three auxiliary bounds — on the softmax map, on the responsibilities, and on the Q-function — used to analyse the limit points in Part (b) of the main proof; they compare the surrogate-based and the true responsibilities.

Lemma 2 (Softmax stability). *Let $a_k, \tilde{a}_k > 0$ satisfy $e^{-\varepsilon} a_k \leq \tilde{a}_k \leq e^{\varepsilon} a_k$. Then the normalised vectors $p_k = a_k / \sum_j a_j$ and $\tilde{p}_k = \tilde{a}_k / \sum_j \tilde{a}_j$ obey $|\tilde{p}_k - p_k| \leq (e^{2\varepsilon} - 1) p_k \leq 3\varepsilon p_k$ for $\varepsilon \leq 1$.*

Proof. $\tilde{p}_k \leq a_k e^{\varepsilon} / \sum_j a_j e^{-\varepsilon} = p_k e^{2\varepsilon}$ and similarly $\tilde{p}_k \geq p_k e^{-2\varepsilon}$. $\square$

Lemma 3 (Responsibility deviation). *Let $w_{mk}(\theta) = q_{mk}^*(\theta)$ be the true-surrogate responsibilities (Proposition 1 with $u = u_{\text{true}}$) and $q_{mk}^{(t)}$ the surrogate responsibilities of the E-step at iteration t . For $\varepsilon_t \leq 1$, $|q_{mk}^{(t)} - w_{mk}(\theta^{(t)})| \leq 3\varepsilon_t w_{mk}(\theta^{(t)})$.*

Proof. Apply Lemma 2 with $a_{mk} = r_{mk}^{(t)} e^{-u_{\text{true}}(\alpha_k^{(t)}, \mathbf{r}_k^{(t)})}$ and $\tilde{a}_{mk}$ obtained by replacing u_{true} by $u^{(t)}$. The ratio $\tilde{a}_{mk}/a_{mk} = e^{u_{\text{true}} - u^{(t)}}$ satisfies $e^{-\varepsilon_t} \leq \tilde{a}_{mk}/a_{mk} \leq e^{\varepsilon_t}$ by (15), so Lemma 2 applies with $\varepsilon = \varepsilon_t$. $\square$

Lemma 4 (Q-function deviation). *Define the surrogate Q-function*

$$\tilde{Q}^{(t)}(\theta; \theta') = \sum_{m=1}^M |\mathcal{C}_m| \sum_{k=1}^K q_{mk}^{(t)}(\theta') [\log r_{mk} - u^{(t)}(\alpha_k, \mathbf{r}_k)].$$

Under Assumption 1 there exists a constant $C_Q = C_Q(N, K, U_{\max})$ such that for every θ, θ' and every t with $\varepsilon_t \leq 1$, $|\tilde{Q}^{(t)}(\theta; \theta') - Q_{\text{true}}(\theta; \theta')| \leq C_Q \varepsilon_t$.

Proof. Add and subtract $\sum_m |\mathcal{C}_m| \sum_k q_{mk}^{(t)} [\log r_{mk} - u_{\text{true}}(\alpha_k, \mathbf{r}_k)]$. The first piece, the surrogate-error term, is $\sum_m |\mathcal{C}_m| \sum_k q_{mk}^{(t)} (u_{\text{true}} - u^{(t)})$ and is bounded by $N \varepsilon_t$. The remainder splits as $\sum_m |\mathcal{C}_m| \sum_k (q_{mk}^{(t)} - w_{mk})(-u_{\text{true}})$ and $\sum_m |\mathcal{C}_m| \sum_k (q_{mk}^{(t)} - w_{mk}) \log r_{mk}$. The first is bounded by $3NK U_{\max} \varepsilon_t$ via Lemma 3

(because $|u_{\text{true}}| \leq U_{\max}$). For the second, using $|x \log x| \leq 1/e$ on $[0, 1]$ together with the softmax-stability bound of Lemma 2, one obtains a bound of $3N\varepsilon_t/e$. Summing the three contributions gives the claim with $C_Q = N(1 + 3K U_{\max} + 3/e)$. $\square$

A.2 Main Proof

Proposition 2 (Quasi-monotonicity of L_{true}). *Under Assumptions 1–3, for every t ,*

$$L_{\text{true}}(\theta^{(t+1)}) \geq L_{\text{true}}(\theta^{(t)}) - 2N\varepsilon_t. \quad (20)$$

Proof. We chain five inequalities. First, $L_{\text{true}}(\theta^{(t+1)}) \geq \mathcal{F}_{u_{\text{true}}}(q^{(t)}, \theta^{(t+1)})$ since $L_u(\theta) = \max_q \mathcal{F}_u(q, \theta)$. Second, by Lemma 1, $\mathcal{F}_{u_{\text{true}}}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t+1)}) - N\varepsilon_t$. Third, the GEM Assumption 3 yields $\mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}_{u^{(t)}}(q^{(t)}, \theta^{(t)})$. Fourth, the E-step makes the latter equal to $L^{(t)}(\theta^{(t)})$. Fifth, Corollary 1 gives $L^{(t)}(\theta^{(t)}) \geq L_{\text{true}}(\theta^{(t)}) - N\varepsilon_t$. Combining the five steps yields (20). $\square$

Part (a): convergence of the likelihood. Set $L_{\max} = \sup_{\theta \in \Theta} L_{\text{true}}(\theta)$, which is finite because Θ is compact (Assumption 4 and finiteness of $\mathcal{A}$). Define $V_t = L_{\max} - L_{\text{true}}(\theta^{(t)}) \geq 0$. From Proposition 2,

$$V_{t+1} \leq V_t + 2N\varepsilon_t, \quad \sum_{t=0}^{\infty} 2N\varepsilon_t < \infty \text{ by Assumption 2.}$$

The Robbins–Siegmund theorem [37] (with $a_t = 0$, $b_t = 2N\varepsilon_t$) implies that $\{V_t\}$ converges, hence $\{L_{\text{true}}(\theta^{(t)})\}$ converges. $\square$

Part (b): stationarity of limit points.

Step 1: existence. Θ is compact, so $\{\theta^{(t)}\}$ has at least one limit point $\theta^* = \lim_{j \rightarrow \infty} \theta^{(t_j)}$.

Step 2: contradiction setup. Suppose θ^* is not stationary: there is $\theta' \in \Theta$ and $\delta > 0$ with $Q_{\text{true}}(\theta'; \theta^*) - Q_{\text{true}}(\theta^*; \theta^*) = \delta > 0$.

Step 3: continuity. The map $\theta' \mapsto Q_{\text{true}}(\theta; \theta')$ is continuous on Θ : $w_{mk}(\theta')$ is continuous because u_{true} is continuous in $\mathbf{r}$ (Assumption 4) and softmax is continuous; and the product $w_{mk} \log r_{mk}$ extends continuously to $r_{mk} = 0$ because $w_{mk} = O(r_{mk})$ and $r \mapsto r \log r$ is continuous at 0. Thus, for j large enough, $Q_{\text{true}}(\theta'; \theta^{(t_j)}) - Q_{\text{true}}(\theta^{(t_j)}; \theta^{(t_j)}) \geq \delta/2$.

Step 4: transfer to $\tilde{Q}$. By Lemma 4,

$$\tilde{Q}^{(t_j)}(\theta'; \theta^{(t_j)}) - \tilde{Q}^{(t_j)}(\theta^{(t_j)}; \theta^{(t_j)}) \geq \delta/2 - 2C_Q \varepsilon_{t_j} \geq \delta/4$$

for j large enough, since $\varepsilon_{t_j} \rightarrow 0$.

Step 5: GEM ascent. Because the M-step searches over a candidate set that contains θ' (by finiteness of $\mathcal{A}$ and the closure properties of the $\mathbf{R}$ -update), $\tilde{Q}^{(t_j)}(\theta^{(t_j+1)}; \theta^{(t_j)}) \geq \tilde{Q}^{(t_j)}(\theta'; \theta^{(t_j)})$. Translating back through Lemma 4 once more,

$$\Delta_{t_j} := Q_{\text{true}}(\theta^{(t_j+1)}; \theta^{(t_j)}) - Q_{\text{true}}(\theta^{(t_j)}; \theta^{(t_j)}) \geq \delta/4 - 2C_Q \varepsilon_{t_j} \geq \delta/8$$

for j large enough.

Step 6: summability contradiction. The Gibbs inequality $L_{\text{true}}(\theta^{(t+1)}) - L_{\text{true}}(\theta^{(t)}) \geq \Delta_t$ together with Proposition 2 implies $\Delta_t \geq -2N\varepsilon_t$. Hence the non-negative sequence $\Delta_t + 2N\varepsilon_t$ satisfies $\sum_t (\Delta_t + 2N\varepsilon_t) < \infty$ by Robbins–Siegmund applied to V_t , and consequently $\sum_t \Delta_t < \infty$. But Step 5 gives $\Delta_{t_j} \geq \delta/8$ for infinitely many j , yielding $\sum_t \Delta_t = +\infty$ — a contradiction. Therefore θ^* is stationary. $\square$

A.3 Convergence Rate

A by-product of the proof of Theorem 1(a) is the explicit rate

$$L_{\text{true}}(\theta^*) - L_{\text{true}}(\theta^{(T)}) \leq 2N \sum_{t=T}^{\infty} \varepsilon_t. \quad (21)$$

If $\varepsilon_t = O(t^{-p})$ with $p > 1$, this gives $L_{\text{true}}(\theta^{(T)}) \rightarrow L_{\text{true}}(\theta^*)$ at rate $O(T^{1-p})$, matching the rate observed for stochastic-EM analyses [38].

B Additional Experimental Results

This appendix collects two ablations on the CIFAR-100/SVHN mixture ($K = 2$, $M = 30$ semantic clusters) referenced from Section 5.2. All numbers are cluster-gate test accuracy (mean $\pm$ std over the indicated number of seeds), evaluated under the same protocol as Table 1.

B.1 Decomposition of the cluster-gate accuracy

To attribute the MoE gain to its two ingredients — the cluster $\rightarrow$ expert *routing* and the per-expert *architecture* — we vary one factor at a time (Table 2). Block (i) fixes a single DARTS architecture (searched on the full mixture) and only changes the routing: moving from a random cluster split to the oracle source split raises accuracy by 3.0 percentage points, so the routing is the main lever. Block (ii) fixes the oracle routing and instead changes the architecture: a single full-mix architecture, a CIFAR-searched one and an SVHN-searched one all lie within 1 p.p. of each other, i.e. once the routing is correct, architectural heterogeneity between experts adds little on this benchmark. Block (iii) isolates the contribution of *learning* the routing inside SGEM: freezing the routing matrix at its random initialisation (no M-step on **R**) drops the source-recovery rate to chance (59%) and the accuracy to 0.598, whereas the full method recovers the source partition (92%) and reaches 0.620 — essentially the value obtained with the oracle routing in block (i), but *without* any source labels.

Tab. 2: Ablation decomposing the cluster-gate accuracy. “src. match” is the agreement of the hard routing with the ground-truth source partition; it is 100% by construction whenever the oracle split is used.

Setting	test acc.	src. match
<i>(i) vary routing, architecture fixed (full-mix DARTS)</i>		
random cluster split	0.587 ± 0.006	$\approx 54\%$
oracle source split	0.617 ± 0.007	100%
<i>(ii) vary architecture, oracle routing</i>		
full-mix architecture	0.627 ± 0.009	100%
CIFAR-searched architecture	0.636 ± 0.003	100%
SVHN-searched architecture	0.626 ± 0.006	100%
<i>(iii) learn the routing (SGEM, no labels)</i>		
routing frozen at random init	0.598 ± 0.003	$59 \pm 7\%$
SGEM (full)	0.620 ± 0.009	$92 \pm 2\%$

B.2 Sensitivity to the load-balancing weight

Section 5.2 keeps the load-balancing weight small ($\lambda_{\text{LB}} = 1$) because the correct source partition is itself imbalanced (19/11), so a strong balancing penalty would push the routing away from it. We make this trade-off quantitative by scoring three candidate hard routings under the *penalised* objective $L - \lambda_{\text{LB}} \cdot \text{LB}_N$ actually optimised in the M-step, where the per-expert losses u_k are obtained by genuinely training a fixed architecture on each expert’s clusters (no surrogate), $L = -\sum_k N_k u_k$ is the resulting size-weighted objective, and $\text{LB}_N = K \sum_k P_k^2 \cdot N$ is the (extensive) balancing penalty (Table 3).

Tab. 3: Penalised objective of three hard routings on real-trained per-expert losses. The penalised score is $L - \lambda_{LB} LB_N$.

hard split	u_k (per expert)	L	LB_N
oracle [19, 11]	[2.73, 0.51]	-48099	31491
balanced, SGEM [15, 15]	[1.91, 1.90]	-55973	29400
balanced, random [15, 15]	[1.77, 2.15]	-57726	29400

On the unpenalised objective ($\lambda_{LB} = 0$) the oracle split is preferred by a margin of $\Delta L = 7874$. However, because the oracle split is imbalanced it is penalised more heavily than a balanced one by $\Delta LB_N = 2091$ per unit of λ_{LB} . The oracle split therefore wins the penalised objective only while $\lambda_{LB} < \Delta L / \Delta LB_N \approx 3.77$; beyond this cross-over a balanced (and hence domain-blind) split scores higher. This is a direct, surrogate-free conflict between load balancing and the imbalanced domain structure, and explains why a large λ_{LB} — which reliably prevents collapse — would also suppress the very specialisation we seek; we therefore operate well below the cross-over.